

Basic (trace the loop on paper) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (a precise, reasoned trace, and the trickier cases a real agent hits). Read Basic first even if this feels easy — it's the fastest way to spot exactly what Intermediate adds.

- [Hint 1 — The idea, and picking the right task](#)
- [Hint 2 — Writing the full trace](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and picking the right task

Basic Version

An agent loop is just: ask the model what to do → do it → show the model what happened → ask again, until it answers instead of asking for another tool. Pick a task that needs 2 tools where the second call's input depends on the first call's output — like "what's the weather in Paris, then convert that temperature to Fahrenheit." Write out each round of the loop by hand before touching your editor.

Intermediate Version

The task has to be genuinely sequential, not just "2 things to do." "What's the weather in Paris, and what's the weather in Tokyo?" is 2 independent calls — either could go first, or even run at once. "What's the weather in Paris, in Fahrenheit?" is different: you cannot call the conversion tool until you already have the Paris temperature, because its input *is* the first tool's output. That dependency is what makes this a *loop* instead of just "2 tool calls" — write your task, then check it has this property before tracing anything.

Two traps worth planning for once you're comfortable with the basic shape: **a task that looks like it needs a tool but doesn't** (e.g., "what's 15% of the Paris temperature, if it's 20°C?" — once you already know it's 20, the arithmetic needs no tool at all), and **a tool that could plausibly fail** (what would the model's next Thought look like if `get_weather("Paris")` returned an error instead of a temperature?). Just notice, before picking your task, that "which tool, and when" is a real decision the model has to get right, not a given — these are the 2 failure modes the rest of this document is built around.

Difference between Basic and Intermediate: Basic gives the loop's shape and a ready-made example task. Intermediate makes precise *why* that task belongs in this exercise — the sequential dependency is what makes it a loop and not just 2 independent calls — and previews the 2 failure modes worth watching for: an unneeded tool call, and a tool that fails.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — Writing the full trace

Basic Version

Use this template for every round:

```
Thought: [what the model is thinking]
Action: [tool name] with [arguments]
Observation: [what the tool returned]
```

and close with:

```
Thought: I have enough information now.
Final Answer: [the answer]
```

Trace the Paris weather + Fahrenheit task, round by round, using this exact template.

Intermediate Version

Be precise about *why* each Thought leads to that specific Action — this is the part people skip, and it's the part that actually builds the mental model. "Thought: I need the current temperature in Paris before I can convert it, so I can't answer yet" names what's missing and why the next Action follows from it. "Thought: I'll check the weather" with no reasoning attached does not. Now trace a second task — "Convert 100 US dollars to euros, then tell me if that's enough for a 90-euro hotel room" — twice: once the correct way (1 tool call, then reasoning over numbers you already have), and once the wasteful way (an unnecessary second "compare numbers" tool call). Comparing your 2 traces side by side is the point, not just producing 1 correct one.

One more worth tracing before you check the Solution: the weather tool call from round 1 fails (`get_weather("Paris")` returns `"Error: service unavailable"` instead of a temperature). Write the Thought that should follow a failed Observation — does the model retry, try something else, or give up and say so honestly? This is deliberately hard to get "right" on paper; the point is noticing that a failed Observation needs its own Thought, not something the loop can just skip over as if it never happened.

Difference between Basic and Intermediate: Basic produces one correct, complete trace using the template. Intermediate adds *reasoned* Thoughts (not just labels), a second trace pair — correct vs. wasteful — so you can see an unnecessary tool call on paper before you ever see one in real

output, and a failed Observation traced honestly — exactly what Core Concepts' "ignoring a failure" problem and the Build Task's recovery requirement are both about.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)